

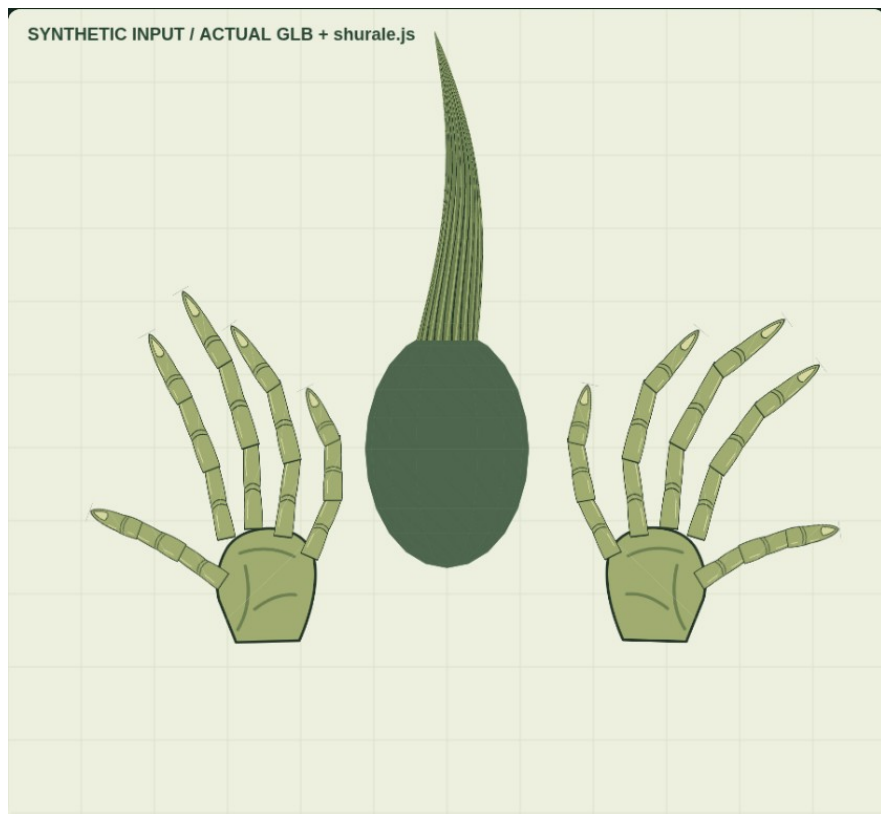
Платформа ушла. Рог остался.

Шүрэле: маленькая книга о некротехнологиях

Проект Eljah/shueraele. Издание 0.2.0, сентябрь 2026.

Рог, две рисованные кисти, JavaScript и инструкция, позволяющая понять эффект даже тогда, когда его платформа больше не помогает разработчику.

Статус издания: исходники и импортируемая графика подготовлены; математическая логика, адаптер на тестовых двойниках и автономный графический стенд проверены. Meta Spark Studio и его распознавание не запускались. Сохранённого редактором `.arproj` в этом издании нет.



Настоящий рендер моделей комплекта, синтетическая поза. Это не камера и не Meta Spark.

1. Что мы здесь воскрешаем

Обычная инструкция по AR начинается с кнопки «Создать эффект». Наша начинается с вопроса: что останется от эффекта, когда этой кнопки больше нет? У «Шүрәле» можно сохранить рисунок, геометрию, правила движения, связь с координатами головы и кисти, описание материалов и испытания. Каталог эффектов, аккаунт и сервер публикации — другие части системы. Их наличие нельзя получить хорошей организацией папок.

Поэтому некротехнология здесь — не обход закрытого сервиса. Это воспроизводимая инженерная реконструкция. Мы отделяем собственную работу от чужого runtime, а подтверждённый результат — от правдоподобного предположения. Старый редактор может оказаться доступен, а может остановиться на авторизации. В обоих случаях исходники не должны превращаться в непонятный архив.

Замысел намеренно небольшой: один длинный рог на голове и две графические кисти с длинными пальцами. Рог следует за головой. Кисть следует за положением руки, а её пальцы живут собственной анимацией. Это театральная перчатка, а не медицински точная цифровая копия ладони. Такой художественный договор нужно сообщать заранее, а не после демонстрации.

Самая полезная вещь в комплекте — не эффектный кадр. Это возможность ответить: какой файл создал этот рог, какая функция согнула палец и какой тест проверил исчезновение руки? Хорошая некротехнология оставляет маршрут назад от результата к исходнику.

2. Четыре разных уровня готовности

Первый уровень — ассеты. Существует ли настоящая сетка, можно ли прочитать её вершины, содержится ли внутри нужная текстура? У нас рог и кисти сохранены в GLB, а рисунки — также в SVG и PNG. Контейнер GLB и иерархия узлов описаны стандартом glTF [S1]. Файл модели не равен проекту Spark: это только его импортируемая часть.

Второй уровень — поведение. Функция `SMotion.pose` выдаёт углы, а адаптер распределяет их по узлам и привязывает корни к сигналам. Эти операции проверяются отдельно от нейросети. Третий уровень — нативное выполнение: конкретный Spark импортирует файлы, находит модули, получает реальные треки и рисует их правильно. Четвёртый — доставка эффекта пользователю через платформу.

В этом издании проверены первые два уровня и программный просмотр моделей. Третий остаётся открытым. Четвёртый не является целью реконструкции. Нельзя заменить третий уровень красивым вторым, а отсутствие четвёртого — назвать ошибкой нашего скрипта.

Скриншот с надписью `ready` тоже не универсальное доказательство. Он показывает, что конкретная инициализация завершилась в конкретной среде. Поэтому рядом с ним должны быть среда, входные данные и название скрипта. Наши кадры содержат пометку о синтетическом входе; исходники теста позволяют повторить их.

3. Как нарисованы руки и построен рог

Кисть — иерархия плоских карточек. Есть одна ладонь и пять пальцев, в каждом по три звена. Всего 16 маленьких сеток и 17 узлов на кисть с учётом калибровочного корня. Три звена у большого пальца выбраны ради одинакового управления графикой; это художественная структура, не утверждение об анатомии человека.

Каждая карточка содержит два треугольника. Вместе обе руки дают 64 треугольника. Для всех звеньев используется один рисунок-атлас 1024×1024 : ладонь, основание пальца, среднее звено и кончик занимают четыре области. Сетка выбирает нужную область координатами UV. Не следует разрезать атлас на десятки отдельных материалов: это усложнит восстановление без художественной пользы.

Рог, напротив, объёмный. Генератор создаёт 19 поперечных колец, постепенно уменьшает радиус, изгибает ось и закрывает торцы. Получается 604 треугольника при высоте 0.275 метра. На сетку наложен рисунок коры. Небольшой зазор между визуальным основанием и головой исправляется посадкой anchor, а не случайным общим масштабом сцены.

Четвёртая модель — эллипсоид головы на 352 треугольника. Это заготовка окклюдера. В лаборатории он виден, чтобы можно было осмотреть форму; в эффекте ему нужен материал, закрывающий геометрию по глубине без рисования зелёного овала поверх лица. Приблизительный эллипсоид не знает причёску, уши и реальный профиль пользователя.



Те же модели после поворота на 45 градусов. Объём рога и плоскость пальцев различимы.

Геометрические единицы — метры, вверх направлена ось Y. Это собственный согласованный договор проекта, совместимый с координатной моделью glTF [S1]. После импорта в редактор договор необходимо проверить измерением, а не верой в расширение файла.

4. Пальцы движутся, но не читают мысли

Движение начинается с авторской позы раскрытой ладони. Для каждого пальца заданы угол разведения и два небольших сгиба. К этим значениям прибавляются синусоидальные колебания. У соседних пальцев фаза сдвинута, поэтому они шевелятся волной, а не складываются одинаковым веером.

```
phase = 2 * Math.PI * (t * 0.6 + finger * 0.13 + slot * 0.2);
wave = Math.sin(phase);
baseAngle = authoredSplay + intensity * 0.09 * wave;
```

Это иллюстрация формулы; полный и исполняемый вариант находится в `src/motion.js`. Подвижность ограничивается диапазоном от 0 до 1. Ноль фиксирует исходную позу. Время берётся по модулю 600 секунд; выбранные частоты дают целое число циклов на этой границе, что проверяется тестом непрерывности.

Адаптер записывает углы в `transform.rotationZ` каждого из 30 звеньев. Геометрия дочернего звена начинается у конца предыдущего, поэтому изменение одного сустава переносит все последующие. Удаление иерархии при конвертации модели разрушит именно этот механизм.

Рог получает отдельное покачивание малой амплитуды. Основную ориентацию головы задаёт штатный Face Tracker, не синусоида. Это важно: декоративное движение нельзя перепутать с трекингом. Мы не получаем из API измеренные сгибы пальцев и не выдаём заранее написанную анимацию за распознавание.

5. Почему камера — это система координат

Родитель объекта определяет, в какой системе координат интерпретируются его положение и поворот. В адаптере вся кисть получает `hand.cameraTransform`. Следовательно, её `anchor` должен находиться прямо под `Camera`. Добавленный между ними повернутый или масштабированный `Null` применит лишнее преобразование, и маска начнёт уезжать.

У рога другая схема: он вложен в `Face Tracker`. Его координаты описывают смещение относительно головы. Локальный ноль геометрии помещён у основания, поэтому подстройка посадки понятна: перемещаем точку роста, а не центр длинного рога.

Зеркальность тоже должна иметь одного хозяина. Вторая кисть подготовлена как зеркальная композиция; адаптер устанавливает абсолютный масштаб калибровочного корня. Если ещё раз отразить изображение родителем, можно получить исходную сторону вместо зеркальной. Проверяется это не названием «левая», а направлением большого пальца на реальном кадре.

Опубликованный снимок деклараций `HandTracking`, изученный для проекта, содержит `cameraTransform`, `boundingBox` и `isTracked` [S2]. Мы не считаем этот сторонний снимок полной спецификацией всех версий. Он задаёт выбранный нами узкий контракт. Число доступных слотов и их фактическую устойчивость должен подтвердить конкретный редактор.

6. Ошибки тоже являются частью маски

Если лицо потеряно, рог должен исчезнуть, а не остаться висеть на последнем месте. Если пропала кисть, её графика не должна продолжать самостоятельно летать перед камерой. Поэтому видимость задаётся отдельно от геометрии и дополнительно учитывает `isTracked`, когда это поле доступно.

Сначала все `anchors` скрыты. После поиска объектов адаптер проверяет их имена. Если нет хотя бы одного обязательного узла, он сообщает `SN ERROR`, не запускает таймер и сохраняет графику скрытой. Такая остановка полезнее, чем полуисправная картинка: сообщение сразу указывает точный отсутствующий объект.

Если второй слот рук недоступен и вызов `API` выбрасывает исключение, адаптер оставляет первую руку, скрывает вторую и пишет предупреждение. Это контролируемое упрощение эффекта, а не доказательство, что все версии поддерживают две руки. Если `runtime` просто не выдаёт второй трек без исключения, видимость определяется числом реально обнаруженных кистей.

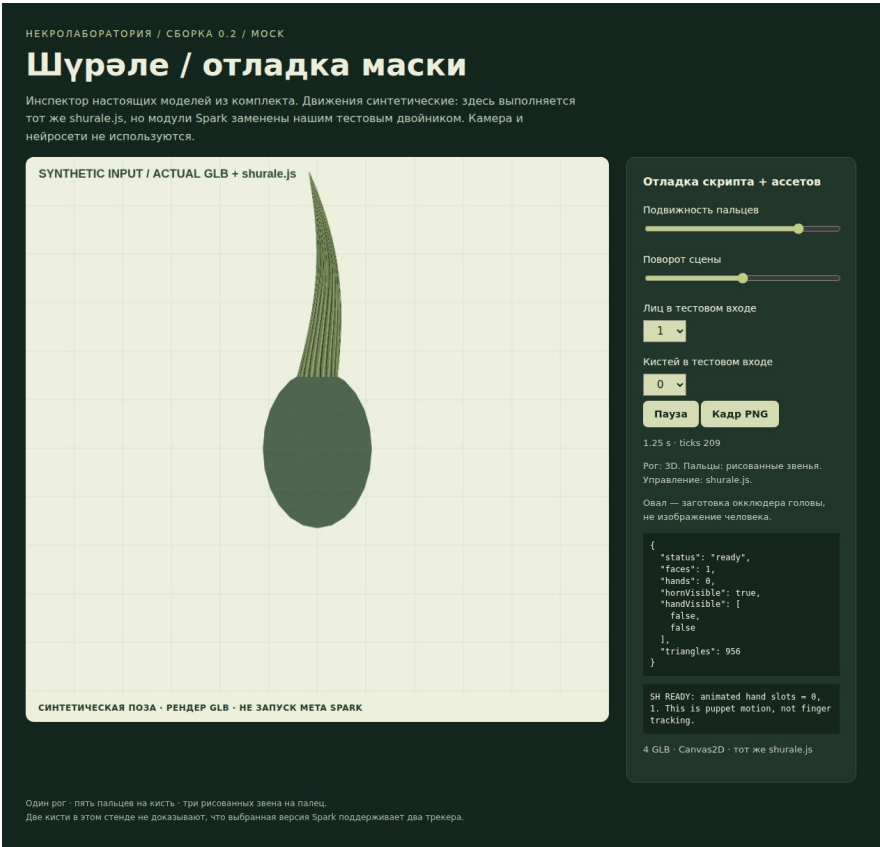
Номер слота не является надёжным именем человека или анатомической стороны. При перекрытии и повторном обнаружении руки могут меняться индексами. В этой версии устойчивое сопоставление личности кисти не реализовано. В документации это должно называться ограничением, а не «редким непредсказуемым поведением пользователя».

7. Что именно показывают отладочные кадры

Лаборатория читает настоящие файлы GLB. Затем выполняет полный собранный `shurale.js`, подставляя наши модули `Scene`, `Time`, `FaceTracking`, `HandTracking` и `Diagnostics`. Сигналы пересчитываются при чтении, поэтому выключение трека меняет уже существующие связи, а не только стартовое состояние сцены.

Тестовый двойник не эмулирует движок Meta. Он знает ровно те методы, которыми пользуется адаптер. Такую границу важно сохранять даже после успешных тестов. В официальной репозитории Meta библиотеки `mocks` и `e2e`-тестов также выделены отдельно [S3]; в нашем проекте их реализация не заимствована.

В подготовительной среде WebGL-контекст не создавался. Вместо него инспектор использует программное проецирование треугольников в `Canvas2D` и наложение текстур. Порядок треугольников задаётся по глубине; это не полноценный `z-buffer` Spark. Ракурс, силуэт, иерархия и изменения позы проверяются, но сложная взаимная окклюзия требует нативного испытания.



После потери обоих синтетических треков рук кисти исчезли; рог остаётся.
Снимок реального браузера.

Скриншоты снимает Playwright с действительно отрисованной страницы [S4]. Рядом сохраняется JSON с входными состояниями, углами и сообщениями скрипта. Сравнение двух изображений дополняется сравнением углов: иначе изменение часов в интерфейсе можно ошибочно принять за анимацию модели.

8. Сборка без шаманства

Для готовой лаборатории достаточно открыть `lab/preview.html`. В ней встроены модели, текстуры, скрипт и тестовый адаптер модулей. Она не обращается к CDN, не просит камеру и не получает данные пользователя. Автономность здесь настоящая, но относится именно к лаборатории, а не к установщику или авторизации Spark.

При изменении исходников выполните генерацию графики, сборку скрипта и сборку HTML. Отдельно запускаются Node-тесты, структурные проверки GLB и браузерный прогон. Команды собраны в README. Для книги есть свой сборщик; PDF требует LibreOffice и системных шрифтов, которые не входят в репозиторий.

В GitHub текстовые исходники служат первичным материалом. Workflow может восстановить PNG, GLB, HTML, DOCX, PDF и скриншоты, затем сохранить их обычным коммитом в рабочую ветку. Читатель получает не ZIP, в котором нельзя просмотреть изменения, а обычные папки. Логи workflow и дерево файлов нужно проверять после выполнения, а не путать YAML с уже сделанным действием.

Для нативной сборки следующий раздел книги даёт паспорт сцены. После импорта особенно важны имена, родитель Camera, альфа и двусторонние материалы. Сначала проверяйте простую позу, затем поворот, только потом сложные перекрытия. Когда неисправны одновременно материалы, координаты и трекинг неисправны, искать причину гораздо тяжелее.

9. Где заканчивается это издание

Эффект ещё не умеет удалять настоящие пальцы из видео и дорисовывать фон. Полупрозрачная или узкая графика покажет исходную руку. Человеческий кулак не превращается в идеально соответствующий ему цифровой кулак: пальцы перчатки продолжают собственную волну. Это свойства выбранной реализации.

Полноценное повторение жестов требует другого источника данных: суставов, устойчивой стороны кисти и обработки перекрытий. Добавление внешней модели в современную веб-камеру создаёт другую реализацию. Она может быть полезной, но нельзя без проверки называть её той же маской для исторического Spark.

Следующий содержательный шаг — открыть комплект в сохранившейся доверенной версии редактора, сохранить штатный проект и записать настоящий прогон. В отчёте должны появиться точная версия, камера, устройство, логи, измерения и дефекты. До этого формулировка «проверено в Spark» остаётся незаполненной строкой, а не красивым заголовком.

Когда-нибудь потеряется и этот браузерный стенд. Поэтому рядом лежат исходные рисунки, генератор сетки, простая математика и объяснение родительских связей. Для следующего разработчика важнее оставить понятные суставы проекта, чем ещё один неподвижный скриншот. Платформа ушла. Рог остался — и понятно, как он устроен.

Приложение А. Паспорт нативной сборки

Это инструкция ручной сборки, не отчёт об успешном запуске Spark. Установщик, редактор и закрытые модели Meta в комплект не входят. Начинать следует с сохранившейся доверенной установки, способной открыть пустой проект. Не отключайте защиту ради случайного «исправленного» инсталлятора.

1. Проверка среды

Запишите точную версию редактора, ОС, источник установщика и его SHA-256. Проверьте возможность открыть пустую сцену и видеть камеру. Ошибка авторизации или отсутствующий runtime — проблема среды; наш JavaScript её не чинит. Не подменяйте сетевые ответы авторизации и не вводите токены в сторонние патчи.

Проверьте по документации именно вашей версии наличие `Scene.root.findFirst`, `FaceTracking.count`, `HandTracking.hand`, `cameraTransform`, `isTracked`, `Time.ms`, `Time.setInterval`. Для эффектов рук включите соответствующие `sarabilities` в настройках, если редактор их требует. Названия панелей менялись; инструкция не выдаёт интерфейс одной версии за все версии.

2. Импорт

Импортируйте четыре файла `spark-import/objects/*.glb`. Если конкретный редактор не читает GLB, конвертируйте в поддерживаемый им формат через доступный 3D-редактор, **сохранив иерархию, имена, метровый масштаб и UV**. Сам факт такой конвертации ещё не доказывает корректность импорта. Не объединяйте все звенья пальцев в одну сетку.

Кисть содержит 17 узлов: калибровочный корень, ладонь и 15 звеньев. Во второй кисти имена начинаются с `SH_hand1_`, а не `SH_hand0_`.

3. Иерархия

Создайте штатный Face Tracker средствами редактора. Его стандартного родителя оставьте на месте (в некоторых проектах это Focal Distance). Назовите трекер SH_face_tracker.

```
Camera
  Focal Distance [если создан редактором]
  SH_face_tracker [штатный Face Tracker]
  SH_horn_anchor [Null]
    SH_horn_geometry [из horn.glb]
  SH_head_occluder [из head_occluder.glb]
  SH_hand0_anchor [Null, ПРЯМОЙ потомок Camera]
  SH_hand0_calibration [из hand_0.glb]
  SH_hand0_palm
  SH_hand0_thumb_0
    SH_hand0_thumb_1
      SH_hand0_thumb_2
  SH_hand0_index_0 / middle_0 / ring_0 / little_0
    соответствующие _1
      соответствующие _2
  SH_hand1_anchor [Null, ПРЯМОЙ потомок Camera]
  SH_hand1_calibration [аналогичная иерархия]
```

Три *_anchor **изначально скрыты**. Это предотвращает показ непривязанных объектов до инициализации или при отсутствии платформенного модуля. После успешной загрузки скрипт управляет их видимостью.

Руки должны быть непосредственно под Camera, потому что адаптер присваивает `root.transform = hand.cameraTransform`. Дополнительный родитель с ненулевым преобразованием исказит результат. Не назначайте этому же свойству одновременно Patch Editor и JavaScript.

4. Рог

Для SH_horn_anchor начальные локальные координаты: X=0, Y=0.125, Z=0.020 метра; масштаб 1; вращения 0. Это отправная точка калибровки, а не универсальная посадка для всех лиц. Рог имеет высоту 0.275 м. Его локальный ноль — у основания, ось роста +Y.

Face Tracker задаёт движение всей головы; скрипт добавляет только небольшое покачивание вокруг локальной Z. Переносите основание рога через anchor, а не редактированием каждого vertex. Проверяйте фронтальный вид, наклон и поворот головы.

5. Материалы

`m_horn_ink`: `horn_bark.png`, плоское/unlit оформление. `m_hand_ink`: `RGBA_hand_atlas.png`, альфа-вырезание примерно 0.35, отображение обеих сторон. GLB содержит текстуры, но настройки импортированного материала всё равно проверьте вручную. Неподдержанное расширение `KHR_materials_unlit` замените эквивалентным штатным материалом.

Объекту `SH_head_occluder` назначьте штатный материал `Occluder` или эквивалентную конфигурацию `depth-only` вашей версии. Он должен закрывать часть рога глубиной, а не рисовать зелёный овал поверх лица. В лаборатории этот объект специально видим как геометрическая заготовка. Его форма приближительна и требует подгонки; причёска им не восстанавливается.

Если обводка кистей исчезает, сначала проверьте UV, прозрачность PNG, режим альфа и `culling`, а не логику трекара. После зеркального масштаба особенно важна двусторонность.

6. Скрипт

Импортируйте только `spark-import/scripts/shurale.js`. Он уже включает `src/motion.js` и `src/spark-driver.js`; не запускайте все три файла одновременно. Этот код не является Node-приложением и не требует установки npm-пакетов в Spark.

В Diagnostics после успешной инициализации ожидается:

```
SH READY: animated hand slots = 0, 1.
```

Реальная строка дополнительно уточняет, что движение пальцев кукольное. При отсутствии второго слота ожидается `SH WARNING`, при отсутствующем узле — `SH ERROR` с его точным именем. `READY` подтверждает только выполнение инициализации, а не качество реального трекинга.

7. Калибровка рук

В `src/spark-driver.js` настройте `handScale` и `handMirror`, затем заново выполните `python tools/build_script.py`. Корни GLB уже содержат зеркальность, а скрипт задаёт абсолютный масштаб: не добавляйте ещё один зеркальный Null без необходимости.

Начните с одной раскрытой ладони лицом к камере. Подгоните размер и расположение калибровочного корня. Проверьте, не перескакивает ли слот с одной руки на другую. Слот 0 не равен анатомически левой кисти; при смене порядка обнаружения стороны могут меняться. Поддержку двух одновременных рук подтвердите отдельным испытанием. Нельзя считать её доказанной только потому, что наш тоск возвращает два объекта.

8. Приёмочный прогон

Снимите кадры в настоящем редакторе: фронтально; поворот головы; одна кисть; две кисти; пальцы в кулаке; рука перед лицом; уход руки; возврат; потеря лица; слабое освещение. Запишите версию runtime, разрешение, устройство, логи, частоту кадров и визуальные дефекты. Лабораторные скриншоты не заменяют эти испытания.

Сохраните штатный .argproj только через редактор, затем закройте и вновь откройте его на копии папки. Проверьте отсутствие абсолютных путей. Если экспорт доступен, сохраните результат отдельно и посчитайте хэш. Не обещайте приёмку современным Instagram: эта работа посвящена исходникам исторического эффекта, а не возвращению закрытого каталога.

Приложение Б. Источники

Дата подготовки: 28 сентября 2026. Собственные проектные решения, размеры и результаты тестов описывают этот репозиторий, а не спецификацию Meta.

1. **Khronos, glTF 2.0 Specification** — нормативная структура GLB, nodes, transforms, materials. Прочитан первоисточник: <https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html> . Наша структурная проверка не является сертификацией Khronos. 2.

Опубликованные TypeScript-декларации Spark:

<https://github.com/Enuriru/spark-ar-types/tree/main/PublicTypes> .

Прочитаны `HandTracking.Hand/index.d.ts` и `HandTracking/index.d.ts`; GitHub сообщает дату последнего изменения этих файлов 2022-05-22. Это сторонняя публикация деклараций, не поставляемый Meta SDK и не доказательство поведения каждой последующей версии. Набор полей `Hand` в этом снимке: `boundingBox`, `cameraTransform`, `isTracked`; модуль предоставляет `count` и `hand(index)`. Наш код намеренно ограничен этим контрактом; отсутствие суставов в данном снимке не означает доказанное отсутствие всех расширенных API во всех версиях Spark. 3. **Meta Incubator, spark-ar-core-libs:**

<https://github.com/facebookincubator/spark-ar-core-libs> . Проверена структура официального репозитория: отдельные библиотеки `mocks` и `e2e tests` существуют. Код оттуда не скопирован; наши двойники написаны для этого проекта и не являются официальной эмуляцией движка. 4. **Playwright Python, Screenshots** — первичная документация метода снимков реального браузера: <https://playwright.dev/python/docs/screenshots> . Скриншоты в `evidence/` создаёт `page.screenshot`; они не нарисованы генеративной моделью. 5. **Meta Spark announcement**

— исторический адрес объявления: <https://spark.meta.com/blog/meta-spark-announcement/> . При текущем обращении страница выдала блокировку, не содержащее объявления. Архивный адрес, использованный ранее в обсуждении:

<https://web.archive.org/web/20250304224509/https://spark.meta.com/blog/meta-spark-announcement/> . Содержимое архива повторно

здесь не проверено. Проект не опирается на предполагаемый работающий сервис публикации.

Не использованы неизвестные инсталляторы, модели распознавания Meta, чужие логотипы, обходы авторизации или закрытые сетевые API. Анатомия и рисунки — авторская стилизация этого комплекта, не реконструкция конкретной исторической иллюстрации Шүрәле.